

- Notes:
- HW3, HW4 graded, solutions posted
 - Opened issue on github repo if score on these parts were not 30/30 or 20/20
 - HW5 posted, due Nov 18

Last class:

- Functions and arguments
- Scope of variables
- default arguments
- *args and **kwargs
- Building data types with classes, attributes and methods

```
In [1]: class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def is_zero(self):
        zero = True
        for val in self.coefficients:
            if val != 0:
                zero = False
                break
        return zero

    def __str__(self):
        terms = []
        deg = self.degree()
        coeffs = self.coefficients

        if not self.is_zero():
            if deg > 1:
                for i in range(deg,1,-1):
                    if coeffs[i] != 0:
                        terms += [f'{coeffs[i]}x^{i}']
            if deg > 0:
                if coeffs[1] != 0:
                    terms += [f'{coeffs[1]}x']
                if coeffs[0] != 0:
                    terms += [f'{coeffs[0]}']
            else:
                terms += ["0"]

        return f"Polynomial p(x) = " + " + ".join(terms)

    def __repr__(self):
        return type(self).__name__ + "(" + repr(self.coefficients) + ")"

    def __eq__(self, other):
        return isinstance(other, Polynomial) and self.coefficients == other.coefficients

    def __add__(self, other):
        if isinstance(other, Polynomial):
            low_degree = min(self.degree(), other.degree())
            scoefs = self.coefficients
            ocoefs = other.coefficients

            coeffs = tuple(sc + oc for sc, oc in zip(scoefs[:low_degree+1], ocoefs[:low_degree+1]))
            coeffs = scoefs[low_degree+1:] + ocoefs[low_degree+1:]

            return Polynomial(coeffs)
        else:
            return NotImplementedError
```

Left off with adding numbers to polynomials:

```
In [2]: p1 = Polynomial((1,2,3))
p2 = Polynomial((7,))
print(p1+p2)

Polynomial p(x) = 3x^2 + 2x + 8
Want to just be able to write p1 + 7
```

```
In [4]: from numbers import Number
class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def is_zero(self):
        zero = True
        for val in self.coefficients:
            if val != 0:
                zero = False
                break
        return zero

    def __str__(self):
        terms = []
        deg = self.degree()
        coeffs = self.coefficients

        if not self.is_zero():
            if deg > 1:
                for i in range(deg,1,-1):
                    if coeffs[i] != 0:
                        terms += [f'{coeffs[i]}x^{i}']
            if deg > 0:
                if coeffs[1] != 0:
                    terms += [f'{coeffs[1]}x']
                if coeffs[0] != 0:
                    terms += [f'{coeffs[0]}']
            else:
                terms += ["0"]

        return f"Polynomial p(x) = " + " + ".join(terms)

    def __repr__(self):
        return type(self).__name__ + "(" + repr(self.coefficients) + ")"

    def __eq__(self, other):
        return isinstance(other, Polynomial) and self.coefficients == other.coefficients

    def __add__(self, other):
        if isinstance(other, Polynomial):
            low_degree = min(self.degree(), other.degree())
            scoefs = self.coefficients
            ocoefs = other.coefficients

            coeffs = tuple(sc + oc for sc, oc in zip(scoefs[:low_degree+1], ocoefs[:low_degree+1]))
            coeffs = scoefs[low_degree+1:] + ocoefs[low_degree+1:]

            return Polynomial(coeffs)
        elif isinstance(other, Number):
            return Polynomial( (self.coefficients[0]+other,) + self.coefficients[1:])
        else:
            return NotImplementedError
```

```
In [5]: p1 = Polynomial((1,2,3))
print(p1+7)

Polynomial p(x) = 3x^2 + 2x + 8
What about 7 + p1?
```

```
In [6]: print(7+p1)
```

```
-----
TypeError                                 Traceback (most recent call last)
Cell In[6], line 1
----> 1 print(7+p1)

TypeError: unsupported operand type(s) for +: 'int' and 'Polynomial'
```

Fails because integers don't know how to add Polynomials to themselves.

We can't change what integers do, but we can add "right addition" rule to our polynomial class:

```
In [7]: from numbers import Number
class Polynomial:
    """
    Perform operations with polynomials

    Stores a polynomial in terms of its coefficients, ordered from lowest order first,
    so that self.coefficients[i] is coefficient of x^i.
    """
    def __init__(self, coeffs):
        self.coefficients = coeffs

    def degree(self):
        return len(self.coefficients)-1

    def is_zero(self):
        zero = True
        for val in self.coefficients:
            if val != 0:
                zero = False
                break
        return zero

    def __str__(self):
        terms = []
        deg = self.degree()
        coeffs = self.coefficients

        if not self.is_zero():
            if deg > 1:
                for i in range(deg,1,-1):
                    if coeffs[i] != 0:
                        terms += [f'{coeffs[i]}x^{i}']
            if deg > 0:
                if coeffs[1] != 0:
                    terms += [f'{coeffs[1]}x']
                if coeffs[0] != 0:
                    terms += [f'{coeffs[0]}']
            else:
                terms += ["0"]

        return f"Polynomial p(x) = " + " + ".join(terms)

    def __repr__(self):
        return type(self).__name__ + "(" + repr(self.coefficients) + ")"

    def __eq__(self, other):
        return isinstance(other, Polynomial) and self.coefficients == other.coefficients

    def __add__(self, other):
        if isinstance(other, Polynomial):
            low_degree = min(self.degree(), other.degree())
            scoefs = self.coefficients
            ocoefs = other.coefficients

            coeffs = tuple(sc + oc for sc, oc in zip(scoefs[:low_degree+1], ocoefs[:low_degree+1]))
            coeffs = scoefs[low_degree+1:] + ocoefs[low_degree+1:]

            return Polynomial(coeffs)
        elif isinstance(other, Number):
            return Polynomial( (self.coefficients[0]+other,) + self.coefficients[1:])
        else:
            return NotImplementedError

    def __radd__(self, other):
        return self+other
```

```
In [8]: p1 = Polynomial((1,2,3))
print(7+p1)

Polynomial p(x) = 3x^2 + 2x + 8
```

If `obj1 + obj2` fails (because `obj1` doesn't know how to add `obj2` to itself), Python calls `__radd__` on `obj2`, if it exists, to see if `obj2` knows how to add `obj1` with `obj2` on the right. This helps make code more maintainable and allows us to avoid a lot of duplication.

Extending Polynomial class (writing more methods) and testing it is on HW6

Shift our attention to coding style.

When we discussed what made "good code", we emphasized that good code should be both correct and readable.

We've been working to develop habits that lead to correct code.

- **testing!**

What about developing habits that lead to readable code?

Programming style has been of concern to programmers for almost as long as there have been programmers!

"The Elements of Programming Style" was first published in 1974, by Kernighan and Plauger, highlighting 56 "lessons" on programming style, illustrated by examples in Fortran and PL/I, many of which remain valid in more modern languages, like Python.

Many issues of programming style arise in many languages. For example, how should we write variable or function names that are composed of several words?

Example: Imagine a code that takes 2 matrices, A and B, and multiplies them

We could write:

- `MatrixMatrixMultiply(A,B)`
- `matrix_matrix_multiply(A,B)`
- `MATRIX_MATRIX_MULTIPLY(A,B)`

and many more variants. Any of these is fine, if used consistently...

These all are in use and have names:

- Capitalize first letters, no underscores, is `CamelCase`
- all lower case with underscores is `snake_case`
- all uppercase with underscores is `SCREAMING_SNAKE_CASE`

Other things that we think about are:

- indentation (especially in Python)
- vertical alignment of text
- spaces and tabs

What Style rules to follow?

In Python projects, it is common to follow the PEP8 Style Guide. (PEP stands for Python Enhancement Proposal.) While there are other style conventions for Python code, many projects follow PEP8 or very similar rules

What are the rules of PEP8?

1. Use 4 spaces for each indentation level (functions, `for`/`while` loops, etc.)

```
In [9]: def my_function(x):
    return 2*x
```

```
In [10]: def my_function2(x):
    if x>0:
        return x
    else:
        return -x
```

2. Limit all lines to no more than 79 characters, 72 for comments

Python uses 2 methods for line continuation: a) implied continuation for code inside parentheses:

```
In [11]: def my_function(argument1, argument2,
    argument3, argument4):
    print(argument1)
    print(argument2)
    return argument3 + argument4
```

b) Use backslash character to indicate line continuation:

```
In [13]: with open('/Users/smaclachlan/work/teaching/CMSC6950/Fall2025/Notes/pl_power.py') as file1, \
    open('/Users/smaclachlan/work/teaching/CMSC6950/Fall2025/Notes/pl_copy.py' > 'w') as file2:
    file2.write(file1.read())
```

3. Continuation lines should either align vertically with opening parenthesis or use a "hanging indent" (another level of indentation). If hanging, then have no arguments on first line:

```
In [14]: def long_function_name(
    argument1, argument2,
    argument3, argument4):
    print(argument2)

    val = long_function_name('string1', 'string2',
    'string3', 'string4')

    val_alternate = long_function_name(
    'string1', 'string2', 'string3', 'string4')
```

`string2`

`string2`

Note contrast with:

```
In [15]: def long_function_name2(
    argument1, argument2,
    argument3, argument4):
    print(argument2)
```

This is hard to read, because we can't tell where arguments end and code begins

Since "if" is short, sometimes we break these rules to improve readability.

Contrast:

```
In [16]: def logical_function(logical_statement1,
    logical_statement2):
    if (logical_statement1) and
    logical_statement2):
        print("cat")
    else:
        print("dog")
```

This is hard to read, as indent for hanging bracket is aligned with start of code. So, preferred as:

```
In [17]: def logical_function(logical_statement1,
    logical_statement2):
    if (logical_statement1
    and logical_statement2):
        print("cat")
    else:
        print("dog")
```

Easier to see what is in conditional, and what is outside of it

We also consider these indentations when defining lists:

```
In [18]: my_list = [
    1, 2, 3,
    4, 5, 6
    ]
```

Can align that closing bracket with hanging indent or with start of name:

```
In [19]: my_list2 = [
    1, 2, 3,
    4, 5, 6
    ]
```

Same rules for dictionaries

4. No tabs! Always use spaces for indentation. Most editors automatically translate tabs into spaces. Make sure that yours does!

Note: Cannot mix tabs and spaces for indentation in Python code!

Align arithmetic operators at start of lines:

```
In [20]: val1 = 1
val2 = 2
val3 = 3
val4 = 4
val5 = 5
my_calculation = (val1
                  + (val2 - val3)
                  - val4
                  + val5)
```

Easier to verify correctness in this form vs.

```
In [21]: my_calculation = (val1 +
    (val2 - val3) -
    val4 +
    val5)
```

6. Use blank lines to indicate structure:

- 2 blank lines before top-level function and class definitions, or between 2 top-level functions or classes
- single blank line before/between methods inside of a class
- extra blank lines either to
 - separate groups of related functions
 - inside longer functions, to indicate "sections" of the code

7. Each imported module goes on a separate line

```
In [22]: import numpy
import pandas
```

Not:

```
In [23]: import numpy, pandas
```

Avoid wildcard imports:

```
In [24]: from numpy import *
```

As these lead to namespace conflicts. Better to do:

```
In [25]: import numpy as np
```

or

```
In [26]: from numpy import load
```

8. Use only either singles quotes or double quotes to indicate strings. And use the other for quotes inside a string

```
In [27]: my_string = 'this is a string "containing a quotation"'
print(my_string)

this is a string "containing a quotation"
```

Note:

```
In [28]: my_string = 'this is a string 'containing a quotation''

Cell In[28], line 1
my_string = 'this is a string 'containing a quotation''

SyntaxError: invalid syntax
```

Would need to write:

```
In [29]: my_string = 'this is a string \'containing a quotation\''
print(my_string)

this is a string 'containing a quotation'
```

This works, but is harder to read.

9. Avoid extra spaces:

```
In [30]: my_dict = {'frogs': 2, 'cats': 3}

is preferable to
```

```
In [31]: my_dict = { 'frogs': 2 , 'cats': 3 }
```

Similarly:

```
In [32]: x = 3
if x == 4: print(x)

is preferable to
```

```
In [33]: x = 3
if x == 4 : print(x)
```

Always surround binary operators (`=`, `>`, `<`, `and`, ...) by 1 space on either side, use symmetric spacing for clarity in other instances

```
In [34]: x = 3
y = 4
c = x*x + y*y
```

Exception: don't use spaces around `=` when declaring default values in a function:

```
In [35]: def my_square(x=0):
    return x*x
```

10. Comments should be complete sentences, clear and understandable. Use inline comments sparingly

```
In [36]: # Increment x by 1
x = x+1
```

Preferable to

```
In [37]: x = x+1 # Increment
```

11. Naming conventions:

- modules have short, all-lowercase names
- use underscores in module names if needed
- class names should use CamelCase (CapWords)

- Exceptions use CamelCase
- functions and variables should be snake_case (lowercase_with_underscores)
- same for methods. For non-public methods, use leading underscore in name

And a few more...

This is a lot of rules! How do we make sure that we follow all of them???

Install `flake8`, like `pytest`, this checks our python code for conformance to PEP8 standards

```
[CMSC6950_venv] [math-scott02:teaching/CMSC6950/Fall2025] smaclachlan$ cd HW/CMSC6950_HW6_Classes/
polynomial.py:2:1: E302 expected 2 blank lines, found 0
polynomial.py:3:1: W293 blank line contains whitespace
polynomial.py:4:22: E231 missing whitespace after ','
polynomial.py:22:24: W291 trailing whitespace
polynomial.py:23:35: E231 missing whitespace after ','
polynomial.py:23:37: E231 missing whitespace after ','
polynomial.py:33:16: F541 f-string is missing placeholders
polynomial.py:36:35: E225 missing whitespace around operator
polynomial.py:38:20: E341 missing whitespace after ','
polynomial.py:39:80: E501 line too long (87 > 79 characters)
polynomial.py:41:21: E231 missing whitespace after ','
polynomial.py:42:28: E231 missing whitespace after ','
polynomial.py:43:43: E231 missing whitespace after ','
polynomial.py:47:41: E231 missing whitespace after ','
polynomial.py:47:73: E231 missing whitespace after ','
polynomial.py:47:80: E501 line too long (96 > 79 characters)
polynomial.py:50:30: E231 missing whitespace after ','
polynomial.py:51:80: E501 line too long (82 > 79 characters)
polynomial.py:55:22: E231 missing whitespace after ','
polynomial.py:58:22: E231 missing whitespace after ','
polynomial.py:64:21: E231 missing whitespace after ','
polynomial.py:70:22: E231 missing whitespace after ','
[CMSC6950_venv] [math-scott02:Fall2025/HW/CMSC6950_HW6_Classes] smaclachlan$
```

That's a lot of errors in style! Let's fix them...

Look at list here:

1. No blank lines before class declaration. (Fix this, run `flake8` again, error disappears.)
2. Whitespace on blank line. Remove it
3. Add space after all commas
4. Line 24, space after :
5. Line 38, space around + operator
6. Split lines 47/53
7. Don't want to split line 50, so annotated with `# noqa E501`, which tells PEP8 to ignore this
8. Remove unneeded `f` on string

With that, code passes `flake8` check

Running `flake8` at command line catches PEP8 mistakes after we make them. To catch them as we make them, install a "linter" (name is meant to suggest lint filter on clothes dryer... we "lint" our code to get rid of these bits of "lint")

Can install a linter in VSCode. This will actively highlight mistakes in our code as we make them.

DO THIS!

Many projects build `flake8` (or similar) style checks into their code review process, so that no PRs can be merged until style testing passes. Starting with Problem 2 on HW6, this will be the case for your code as well!

Following consistent coding style helps us read our code, both to understand what correct code is doing and to diagnose (and fix!) incorrect behaviour